LATVIJAS UNIVERSITĀTE
EKSAKTO ZINĀTŅU UN TEHNOLOĢIJU FAKULTĀTE
MATEMĀTIKAS NODAĻA

PERFEKTO POLIMONDU EKSISTENCE

KURSA DARBS

Autors: Marta Rudzāte
Studenta apliecības nr.: mr18118
Darba vadītājs: prof. Dr. math. Andrejs Cibulis

RĪGA 2024

ANOTĀCIJA

Anotācija latviešu valodā

Atslēgas vārdi: izogoni, goligoni, polimino, polimondi, rūtiņu režģis, trijstūru režģis.

ABSTRACT

English abstracts

Keywords: .

SATURA RĀDĪTĀJS

Definīcijas **3**

Ievads **4**

1. Pāra skaita malu figūras **5**

1.1. Izogoni un goligoni 5

1.2. Polimino un polimondi 6

2. Nepāra skaita malu polimondi **11**

2.1. Polimondu iegūšana 11

2.2. Konstrukcijas shēmu apraksts 12

2.3. Pārējās konstrukcijas 15

Secinājumi **21**

Izmantotā literatūra un avoti **22**

Pateicības **23**

Pielikumi **24**

A. “Backtracking” algoritms **24**

B. Polimondu problēmas kods **25**

DEFINĪCIJAS

1. definīcija. Par polimino (angl. polyomino) sauc plaknes figūru, kas sastāv no vienības kvadrātiem, kuri pievienoti viens otram pa vesela garuma malām.

2. definīcija. Par polimondu (angl. polyiamond) sauc plaknes figūru, kas sastāv no regulāriem vienības trijstūriem, kuri pievienoti viens otram pa vesela garuma malām.

3. definīcija. Ja daudzstūra (polimino vai polimonda) visi malu garumi, sākot ar vienu virsotni, iet pēc kārtas augošā vai dilstošā secībā, tad tādu daudzstūri sauc par perfektu.

4. definīcija. Par 90 grādu secīgu izogonu (angl. serial isogon of 90 degrees) sauc daudzstūri, kurš satur tikai taisnus leņķus, un kura malu garumi, sākot no kādas virsotnes, ir naturāli skaitļi augošā secībā no 1 līdz n. Šādu daudzstūri arī sauc par goligonu (angl. golygon).

IEVADS

Kursa darbā tiek apskatīti perfektie polimondi, par kuriem zinātniskajā literatūrā ir pieejama ļoti ierobežota informācija. 1988. gadā Lī Salovs (Lee Sallows) definējis 90 grādu secīgu izogonu. Visi polimino ir arī izogoni, tomēr ne visi izogoni ir polimino. Ar polimino ir iespējams formulēt visdažādākos uzdevumus, kurus nevar formulēt izogoniem, kā, piemēram, apskatīt jautājumu par maksimālo un minimālo laukumu, kā arī, vai ar polimino ir iespējams pārklāt plakni.

Pirmais jautājums, kurš rodas pēc perfektu polimino un polimondu definēšanas, ir viņu eksistence jebkuram malu skaitam. Atbildi uz šo jautājumu ir daļēji devuši Lī Salovs kopā ar Martinu Garneru (Martin Gardner), uzrādot konstrukciju perfektiem polimino. Latvijas Universitātes pasniedzēja Elīna Buliņa savā bakalaura un maģistra darbā ir uzrādījusi konstrukcijas perfektiem polimondiem, kuru malu skaitu var izteikt ar $8k$ un $8k - 2$, kā arī ATEE 2022. gada konferencē Elīna Buliņa ar Andreju Cibuli ir devuši metodi, kā konstruēt perfektu polimondu katram pāra malu skaitam $n \geq 6$.

Darba mērķis ir pierādīt eksistenci perfektiem polimondiem ar nepāra malu skaitu $n \geq 5$. Mērķa sasniegšanai tika izvirzīti šādi darba uzdevumi:

1. iepazīties ar literatūru, kas pieejama par perfektajiem polimino un polimondiem;
2. izveidot datorprogrammu, kas izveido visus perfektos polimondus dotam malu skaitam;
3. atrast konstrukcijas, kas veido perfektos polimondus;
4. pierādīt konstrukciju korektumu.

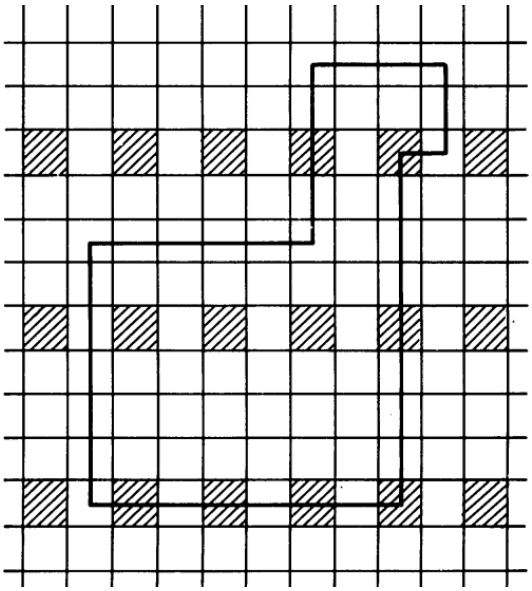
Darbs sastāv no divām nodaļām, definīciju saraksta un diviem pielikumiem. Pirmā nodaļa veltīta pāra skaita malu figūrām: izogoniem, polimino un polimondiem. Bet otrajā nodaļa veltīta nepāra skaita malu perfektiem polimondiem. Darbā iekļauta 1 tabula, 22 attēli, 2 pielikumi un 6 literatūras avoti.

1. PĀRA SKAITA MALU FIGŪRAS

1.1. Izogoni un goligoni

Rakstā [6] L. Salovs definē 90 grādu secīgu izogonu kā ceļu, kurš sākas ar vienu garuma vienību, pēc tam pagriežas $90°$ jebkurā virzienā un iet divas vienības uz priekšu, tad atkal pagriežas $90°$ jebkurā virzienā un tagad jau iet trīs vienības uz priekšu un tā tālāk. Beigās ir jāatgriežas sākumpunktā, veidojot taisnu leņķi ar pirmo posmu. Izogonam ir atļauts sevi krustot, pieskarties stūros un pārklāties posmos. Šajā rakstā ir arī aprakstītas un pamatotas vairākas interesantas lietas par šiem izogoniem.

Algebriski ir pamatots, ka $90°$ secīgam izogonam malu skaitam ir jābūt $8k$, kur $k \geq 1$. To pašu M. Gārners pamato vizuāli ar specifiski iekrāsotu rūtiņu gridu. Metodes ideju var redzēt 1. attēlā, kur jau ir iezīmēts izogons ar 8 malām. Izogona ceļš jāsāk horizontāli no iekrāsotas rūtiņas. Izogons atkārtoti (virziena izvēles brīdī) nokļūs iekrāsotā rūtiņā, kas ir vertikāli sākuma iekrāsotajai rūtiņai, tad un tikai tad, ja malu skaits dalīsies ar 8. Lai pamatotu, ka katram malu skaitam $8k$ eksistē izogons, ir dota konstrukcija. Šī konstrukcija sevi vienmēr krustos, kad $k > 1$. Bet ir dota arī otra konstrukcija, kas veido polimino: Pirmajos $N/4$ posmos iet tikai austrumu un ziemeļu virzienā (pirmajā posmā iet uz austrumiem), tad nākamajos $N/2$ posmos iet tikai rietumu un dienvidu virzienā, atlikušos $N/4$ posmos atkal iet tikai austrumu un ziemeļu virzienā. 1. attēlā ir iezīmēts polimonds, kurš atbilst šai konstrukcijai, kad $N = 8$.



1. att. **Iekrāsots rūtiņu grids ar izogonu**

Donands Knuts (Donald Knuth) šajā pašā rakstā ir devis procedūru, ar kuras palīdzību var

saskaitīt, cik izogonu eksistē, ja fiksē tā posmu skaitu. Šī procedūra ir nedaudz piņķerīga, tāpēc Ričards Kenets Gajs (Richard Kenneth Guy) ir devis formulu, kurā vien ievietojot malu skaitu, uzreiz iegūst atbildi, tomēr tā ir tikai aptuvena.

L. Salovs ir arī pavisam nedaudz apskatījis perfektus polimondus: ir pateikts, ka N stūru polimondi, kuri satur tikai 60 un 300 grādu leņķus, eksistē, ja $N > 9$ un $N \not\equiv 1 \bmod 3$, bet polimondi, kas satur tikai 120 un 240 grādu leņķus, eksistē, ja $N = 6n$.

1990.gada rakstā [5] Aleksandrs Djūdnijs (Alexander Dewdney) 90 grādu secīgu izogonu pārdēvē par goligonu un citiem vārdiem apraksta rakstā [6] dotos pierādījumus.

1.2. Polimino un polimondi

Elīnas Buliņas bakalaura darbā [1] un Valmieras 11. Studentu konferences rakstu krājumā [2] ir detalizēti aprakstīta metode, kā konstruēt perfektu $8k$-stūru polimino, kur $k \geq 1$, kā arī ir pierādīts, ka šī konstrukcija vienmēr dos slēgtu figūru. Šī ir tā pati konstrukcija, kura tika aprakstīta 1991.gada rakstā [6], vien polimondi ir apgriezti spoguļattēlā un par $90°$ pagriezti pulksteņa rādītāja kustības virzienā.

Polimino veidošanas shēma izskatās šādi:

1. izvēlas $n = 8k$,

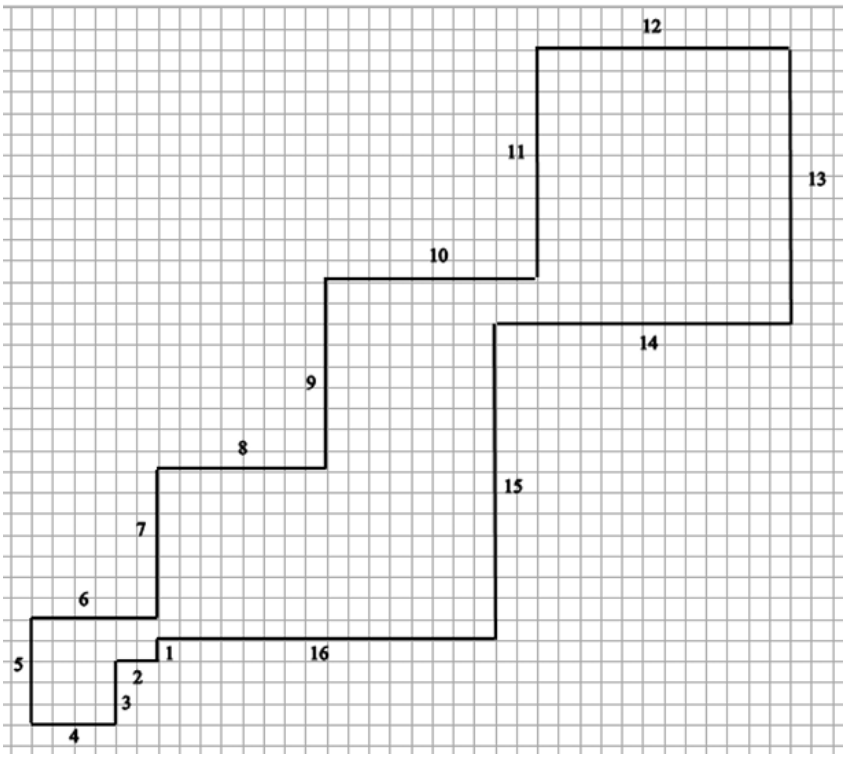
2. malu garumus sadala pa kopām:

   - $H = \{2, 4, ..., 8k\}$
   - $V = \{1, 3, ..., 8k - 1\}$
   - $H_1 = \{2k + 2, 2k + 4, ..., 2k + 4k\}$
   - $H_2 = H \backslash H_1$
   - $V_1 = \{2k + 1, 2k + 3, ...,6k - 1\}$
   - $V_2 = V \backslash V_1$

3. zīmē perfektu polimino:

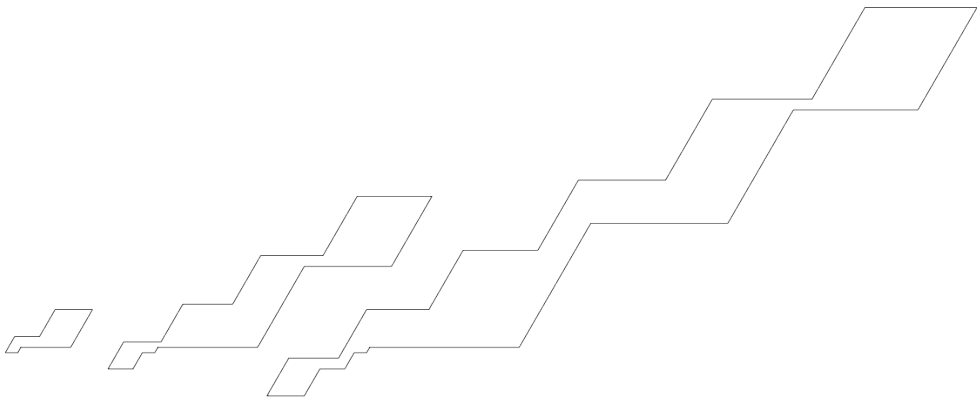
   - malas ar garumiem no kopas H_1 liek uz rietumiem;
   - malas ar garumiem no kopas H_2 liek uz austrumiem;
   - malas ar garumiem no kopas V_1 liek uz dienvidiem;
   - malas ar garumiem no kopas V_2 liek uz ziemeļiem.

2. attēlā ir attēlots 16-stūru perfekts polimino, veidots pēc dotās shēmas.

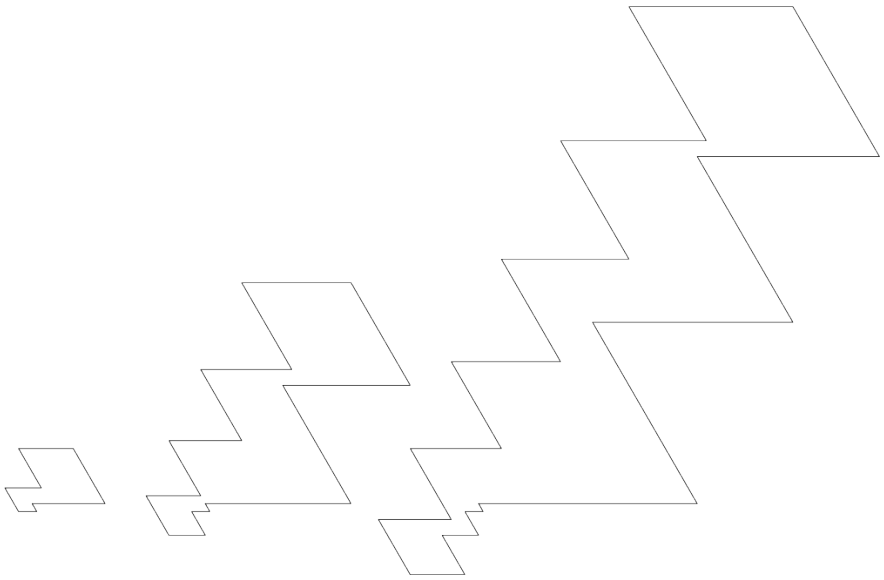


2. att. 16-**stūru perfekts polimino**

Tā kā katru polimino var reducēt uz diviem dažādiem polimondiem, visas vertikālās malas pagriežot par $60°$ pa labi, otrā gadījumā visas vertikālās malas pagriežot par $60°$ pa kreisi, ir iegūtas divas konstrukcijas $8k$-stūru perfektiem polimondiem, kur $k \geq 1$. Šo konstrukciju pirmie polimondi ir attēloti 3. un 4. attēlā.

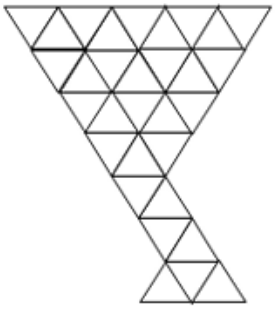


3. att. $8k$**-stūru perfektie polimondi, kur** $k \geq 1$

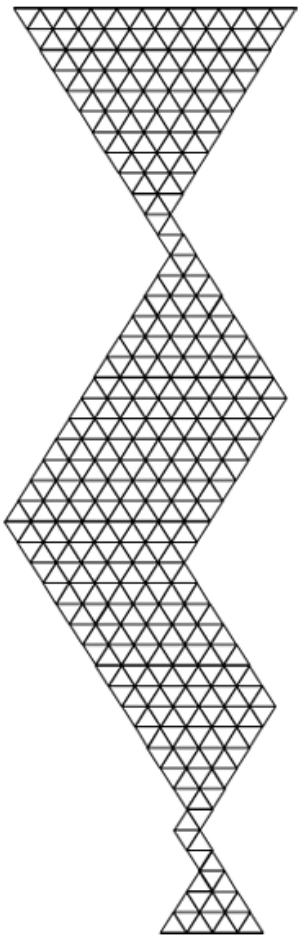


4. att. $8k$**-stūru perfektie polimondi, kur** $k \geq 1$

E. Buliņas maģistra darbā [3] ir dota shēma, kā konstruēt $(8k-2)$-stūru perfektu polimondu, kur $k \geq 1$. 5. un 6. attēlā ir šīs konstrukcijas polimondi, kad k ir attiecīgi 1 un 2.



5. att. 6**-stūru perfektie polimonds**

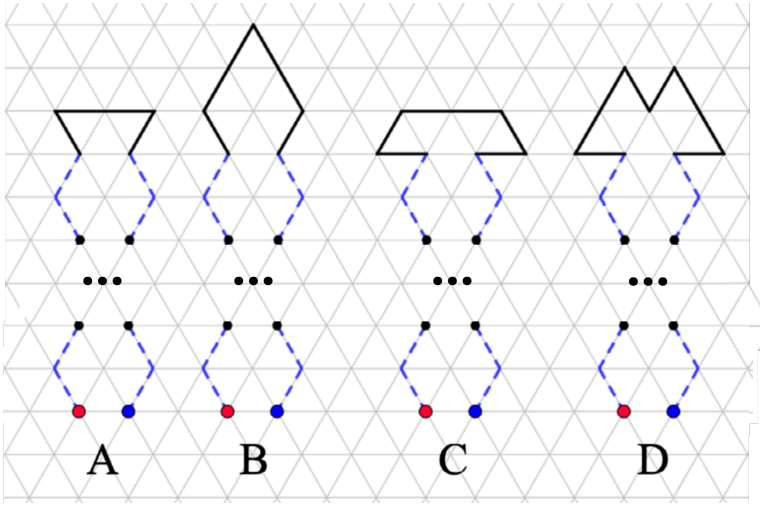


6. att. 14**-stūru perfektie polimonds**

2022. gada ATEE ikgadējās konferences "Būt vai nebūt lieliskam izglītotājam" (angl. To be or not to be a great educator) rakstu krājumā [4] Elīna Buliņa ar Andreju Cibuli ir parādījuši perfektu polimondu eksistenci katram pāra malu skaitam $n \geq 6$.

Polimondi tiek iedalīti četrās kategorijās: $n = 6 + 8k$, $n = 8 + 8k$, $n = 10 + 8k$ un $n = 12 + 8k$, kur $k \geq 0$. Polimonds tiek veidots no divām daļām, 7. attēlā var redzēt daļu veidošanas ideju. Lai izveidotu perfektu polimondu $n = 6 + 8k$ malu skaitam, veido divas A veida lauztas līnijas, priekš perfektiem polimondiem ar $n = 8+8k$, $n = 10+8k$ un $n = 12+8k$ malu skaitu veido attiecīgi B, C un D veida lauztas līnijas. Katrā no lauztajām līnijām jābūt $\frac{n}{2}$ nogriežņiem, tāpēc, skatoties uz 7. attēlu, katra veida lauztajai līnijai ir jāsatur $4k$ raustītos, zilā krāsā iekrāsotos nogriežņus. Beigās šāda veida divas lauztās līnijas savieno kopā līniju galapunktos un, sākot no viena savienojuma punkta, sāk zīmēt malas secīgi ar garumiem no 1 lidz n. Šī metode izmanto invariantu īpašību, galapunkti nemainās, ja katru no malām vienādi daudz pagarina.

6. attēlā var redzēt, kā izskatīsies 14-stūru perfekts polimonds.



7. att. **Pāra skaita malu polimondu veidošana**

2. NEPĀRA SKAITA MALU POLIMONDI

Ar datorprogrammas palīdzību līdz malu skaitam 23 tika saskaitīts perfekto polimondu skaits. Rezultāti ir parādīti 1. tabulā. Priekš lielākām n vērtībām datorprogramma vairs netika darbināta, jo darbības ātrums jau paliek neētiski lēns.

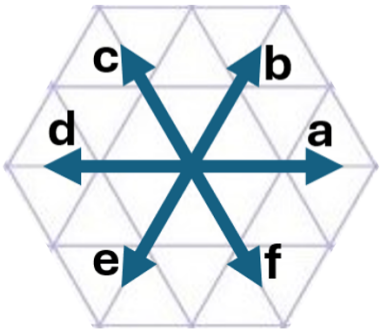
1. tabula

Perfekto polimondu skaits atkarībā no malu skaita

n	5	6	7	8	9	10	11	12	13	14
Skaits	1	1	2	9	3	4	21	87	288	477
n	15	16	17	18	19	20	21	22	23	
Skaits	1335	4026	8599	22326	67068	193445	526694	1464978	4284617	

2.1. Polimondu iegūšana

Ērtības labad izmantosim koordinātu sistēmu, kas attēlota 8. attēlā. Tā kā var pieņemt, ka polimonds vienmēr sākas ar garāko malu un turpinās dilstošā secībā, tad jebkuru perfektu polimondu var aprakstīt ar burtiem a, b, c, d, e, f vienā vienīgā veidā, kā arī katrai burtu virknei atbilsts viens un tas pats polimonds.



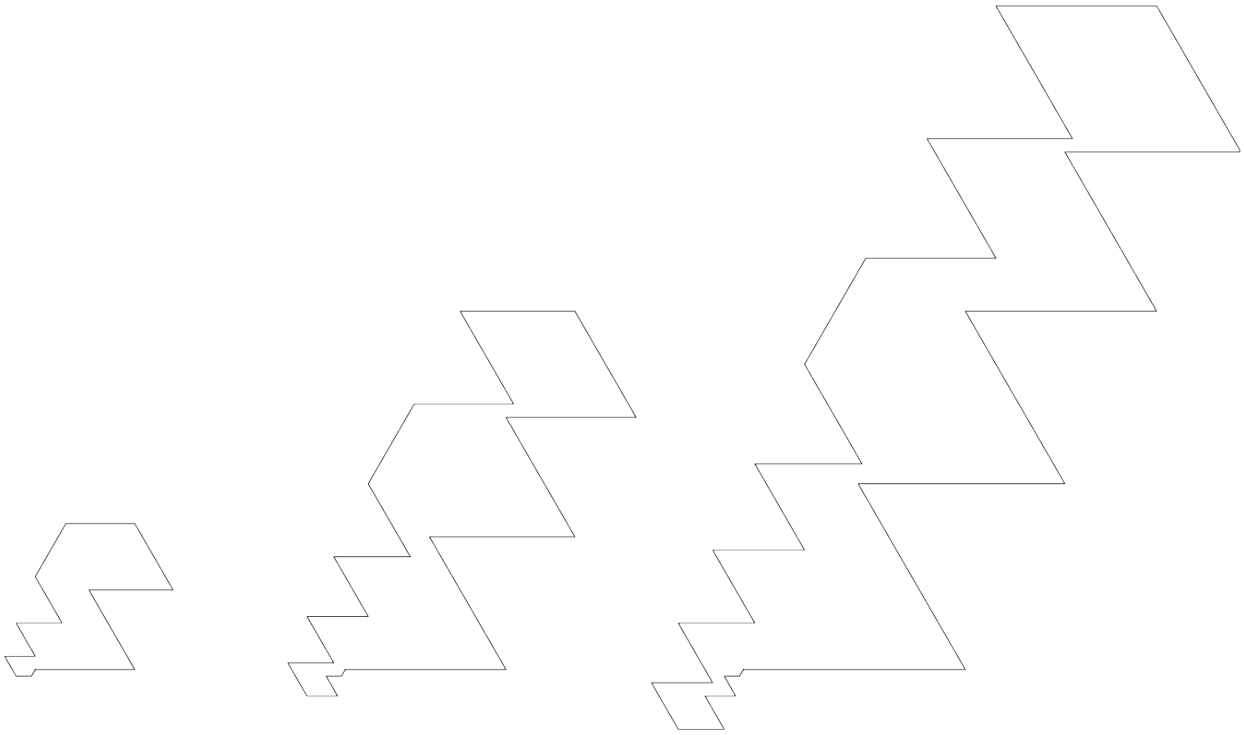
8. att. **Koordinātu sistēma trijstūru režģī**

Kopā ar Kalvi Apsīti tika izveidota datorprogramma, kura dod visus perfektos polimondus, kā arī tika atrasti vairāki koda uzlabojumi, kas paātrina programmas darbību. Viens no labākajiem datorprogrammas uzlabojumiem bija uzrakstīt kodu, kas izmanto "backtracking" algoritmu. Šo algoritmu var daudz kur izmantot, piemēram, sudoku mīklu vai dažādu šaha problēmu risināšanā. Pielikumā ir pievienoti divi datorprogrammu kodi, uzrakstīti valodā Pitons. Pielikumā B ir kods, kas, izmantojot A pielikumā esošo "backtracking" algoritmu, atrod visus

perfektos polimondus dotam malu skaitam N. Ar šo kodu palīdzību tika atrastas visas tālāk parādītās konstrukcijas.

2.2. Konstrukcijas shēmu apraksts

Detalizēti apskatīsim vienu no $(8k-3)$-stūru perfekto polimondu konstrukcijām, kuras pirmie trīs polimondi ir attēloti 9. attēlā.



9. att. **$(8k-3)$-stūru perfektie polimondi, kur $k \geq 2$**

Šo konstrukciju īsā veidā var aprakstīt ar šādu paralēlo bezkonteksta gramatiku:

Aksioma: $aS_1cacdS_2eS_2fdfdfaS_1b$

Nosacījumi: { $S_1 \to caS_1$,

$S_2 \to fdS_2$ }

Zilā krāsā iekrāsotie lielie burti ar indeksu ir ne-termināļi (angl. nonterminals), bet burti melnā krāsā ir termināļi (angl. terminals). Ne-termināļus var aizstāt ar termināļu un ne-termināļu kombinācijām, bet termināļus var uzskatīt kā virzienu apzīmējumus no 8. attēla.

Aksioma ir pats konstrukcijas sākums. Ir divas iespējas, ko ar aksiomu var darīt - izdzēst no tās visus ne-termināļus vai arī katru ne-termināli aizstāt ar nosacījumos norādīto termināļu un ne-termināļu kombināciju.

Ja pirmajā izvēlē izvēlas izdzēst ne-termināļus, tad iegūst termināļu virkni $acacdefdfdfab$. Tā kā tika pieņemts, ka polimonds vienmēr tiks veidots, sākot ar garāko malu, pēc tam ejot dilstoši, tad, pēc 8. attēla apzīmējumiem, šī burtu virkne atbilst tieši mazākajam polimondam 9. attēlā.

Ja pirmajā izvēlē tomēr izvēlas izpildīt nosacījumus, tad iegūst virkni: $acacacS_1dfdS_2efdfS_2dfdfacaS_1b$. Tagad atkal var izvēlēties jebkuru no iepriekš minētajām divām darbībām. Ja izdzēš visus ne-termināļus, tad iegūst vidējo polimondu 9. attēlā. Var arī turpināt pielietot nosacījumus neierobežoti ilgi. Ja n reizes tiek izvēlēts katru aksiomas ne-termināli aizstāt ar nosacījumos norādīto termināļu un ne-termināļu kombināciju, tad beigās, no aksiomas izdzēšot visus četrus ne-termināļus, tiks iegūts perfekts polimonds ar malu skaitu $8(n+2)-3$.

Šāds aksiomas un nosacījumu pielietojums iespējams ir viskompaktākais veids, kas ļoti ātri un viegli iedod burtu virkni perfektam $(8k-3)$-stūru polimondam, kur $k \geq 2$. Tomēr ir arī citi veidi, kā aprakstīt šo konstrukciju. Apskatīsim vienu no šiem veidiem, jo tas palīdzēs pēc tam viegli pamatot, ka šī konstrukcija vienmēr dos polimondus, kuri beigās atgriezīsies tajā pašā punktā, kur sākās.

Izmantosim A, R, ZA, ZR, DA, DR kā saīsinājumus debespusēm, attiecīgi: austrumi, rietumi, ziemeļaustrumi, ziemeļrietumi, dienvidaustrumi, dienvidrietumi. Tos pašus 9. attēlā attēlotos polimondus un visus nākamos šīs pašas konstrukcijas polimondus var iegūt arī pēc šādas shēmas:

1. izvēlas $k \geq 2$,
2. malas ar garumiem no $8k-3$ līdz $6k-2$ pēc kārtas liek uz A, ZR, A, ZR...
3. malas ar garumiem no 6k-3 līdz 4k+1 pēc kārtas liek uz R, DA, R, DA...
4. malu garumā 4k liek DR virzienā.
5. malas ar garumiem no 4k-1 līdz 2k-1 pēc kārtas liek uz DA, R, DA, R...
6. malu garumā 2k-2 liek A virzienā.
7. malas ar garumiem no 2k-3 līdz 1 pēc kārtas liek uz ZR, A, ZR, A...

Tagad pierādīsim, ka polimonds beigās atgriezīsies sākumpunktā.

Pierādījums. Ja ziemeļrietumu un ziemeļaustrumu virzienā likto malu garumu summa būs vie-

nāda ar dienvidrietumu un dienvidaustrumu virzienā likto malu garumu summu, tad sanāk, ka tik vienības, cik polimonds ir pagājis uz augšu, tik pat tas pēc tam ir pagājis uz leju.

Ja austrumu virzienā likto malu garumu summa un ziemeļaustrumu un dienvidaustrumu virzienā likto malu garumu pussumma kopā būs vienāda ar rietumu virzienā likto malu garumu summu un ziemeļrietumu un dienvidrietumu virzienā likto malu garumu pussummu kopā, tad ir zināms, ka tik vienības, cik polimonds ir pagājis pa labi, tik pat tas pēc tam ir pagājis pa kreisi.

Lai aprēķinātu uz katru virzienu likto malu garumu summu, izmantosim divas zināmas formulas:

$$1 + 3 + 5 + ... + (2n - 1) = n^2, \tag{2.1}$$

$$2 + 4 + 6 + ... + 2n = n(n + 1). \tag{2.2}$$

No šīm formulām varam iegūt šādas formulas:

$$(2m - 1) + (2m + 1) + ...(2n - 3) + (2n - 1) = n^2 - (m - 1)^2, \tag{2.3}$$

$$2m + (2m + 2) + ... + (2n - 2) + 2n = n(n + 1) - m(m - 1). \tag{2.4}$$

Austrumu virzienā tiek liktas nepāra garuma malas ar garumiem no $8k - 3$ līdz $6k - 1$. Pēc formulas (2.3) varam aprēķināt šo malu garumu summu.

$$(4k - 1)^2 - (3k - 1)^2 = 7k^2 - 2k.$$

Vēl austrumu virzienā tiek liktas pāra garuma malas ar garumiem no 2 līdz $2k - 2$. Pēc formulas (2.4) varam aprēķināt šo malu garumu summu.

$$(k - 1)k = k^2 - k.$$

Kopā austrumu virzienā likto malu garumu summa ir:

$$7k^2 - 2k + k^2 - k = 8k^2 - 3k.$$

Tieši tādā pašā veidā, skatoties uz shēmu un formulām, varam aprēķināt visos pārējos virzienos likto malu garumu summas. Rietumu virzienā likto malu garumu summa sanāk $8k^2 - 7k + 1$. Ziemeļrietumu virzienā likto malu garumu summa sanāk $8k^2 - 5k$. Dienvidaustrumu virzienā

likto malu garumu summa sanāk $8k^2 - 9k + 1$. Ziemeļaustrumu virzienā iet tikai viena mala, kuras garums ir 1 vienība, kā arī dienvidrietumu virzienā iet tikai viena mala, kuras garums ir $4k$.

Saskaitot kopā ziemeļrietumu un ziemeļaustrumu virzienā likto malu garumu summas, iegūst $8k^2-5k+1$. Saskaitot kopā dienvidaustrumu un dienvidrietumu virzienā likto malu garumu summas, arī iegūst $8k^2 - 5k + 1$. Tas nozīmē, ka tik, cik tika paiets uz augšu, tik pat arī tika paiets uz leju, kas nozīmē, ka mala ar garumu 1 beidzas vertikāli tur pat, kur sākas garākā mala.

Saskaitot kopā ziemeļaustrumu un dienvidaustrumu virzienā likto malu garumu pussummu ar austrumu virzienā likto malu garumiem, iegūst $12k^2 - 7.5k + 1$. Tādu pašu summu iegūst arī saskaitot ziemeļrietumu un dienvidrietumu virzienā likto malu garumu pussumma ar rietumu virzienā likto malu garumiem. Tas nozīmē, ka tik, cik tika paiets pa labi, tik pat tika paiets pa kreisi, kas nozīmē, ka mala ar garumu 1 beidzas horizontāli tur pat, kur sākas mala ar garumu $8k - 3$.

Tā kā mala ar garumu 1 gan horizontāli, gan vertikāli beidzas tur pat, kur sākas mala ar garumu $8k - 3$, tad konstruējot polimondu, iegūst slēgtu figūru. □

2.3. Pārējās konstrukcijas

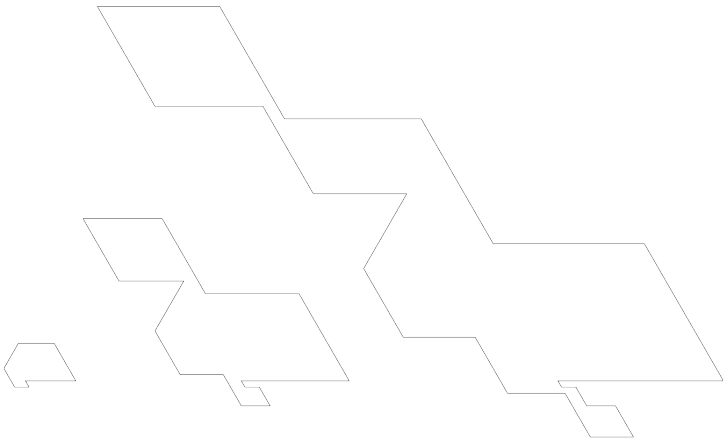
Tieši tādā pašā veidā, kā iepriekšējā sadaļā, var pierādīt arī visām pārējām 13 atrastajām konstrukcijām, ka tās vienmēr dos slēgtas figūras.

Lai apgalvotu, ka šīs konstrukcija tiešām ir korektas un dod perfektus polimondus, vajadzētu arī pierādīt, ka tās sev nekad nepieskaras un sevi nekrusto. Šis pagaidām vēl nav pierādīts. Tomēr vairākām konstrukcijām, vien paskatoties uz pirmajiem perfektajiem polimondiem, uzreiz ir skaidrs, ka polimondi sevi nekad nekrustos un sev nekad nepieskarsies (skatīt, piemēram, 12., 14., 17. un 21. attēlu).

Aksioma: $aS_1cdS_2efaS_2S_1c$

Nosacījumi: { $S_1 \to cdS_1$,

$S_2 \to faS_2$ }

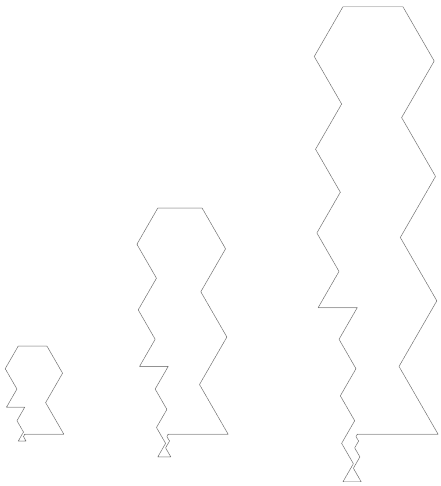


10. att. **($8k-1$)-stūru perfektie polimondi, kur** $k \geq 1$

Aksioma: $aS_1cbcdefeS_2aefeS_2aS_1cb$

Nosacījumi: { $S_1 \to cbS_1$,

$S_2 \to feS_2$ }

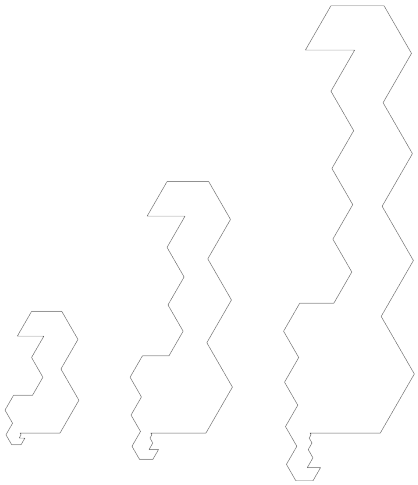


11. att. **($8k-1$)-stūru perfektie polimondi, kur** $k \geq 2$

Aksioma: **a**$S_1bcbcdeaS_2efedS_2efefabdS_1bc$

Nosacījumi: {$S_1 \to bcS_1$,

$S_2 \to efS_2$ }

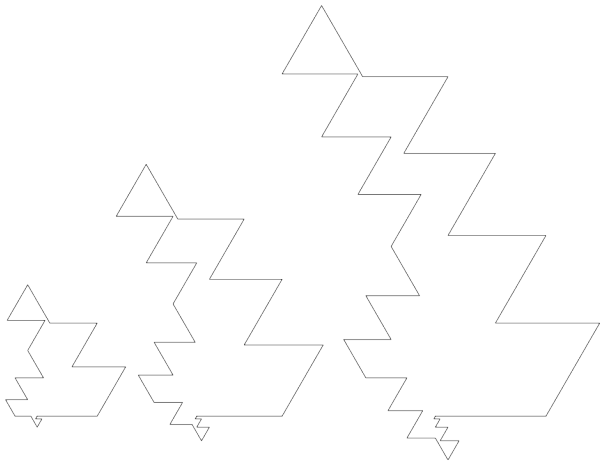


12. att. **($8k-3$)-stūru perfektie polimondi, kur** $k \geq 3$

Aksioma: $aS_1bdbdceaS_2efdfdfaS_2fS_1bdb$

Nosacījumi: $\{S_1 \to bdS_1,$

$S_2 \to eaS_2\}$



13. att. **($8k-5$)-stūru perfektie polimondi, kur** $k \geq 3$

Aksioma: $aS_1bcdfS_2edS_2efacS_1$

Nosacījumi: $\{S_1 \to bcS_1,$

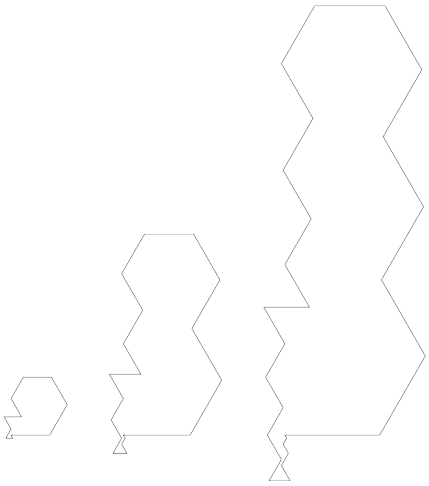
$S_2 \to efS_2\}$

14. att. **($8k-5$)-stūru perfektie polimondi, kur** $k \geq 2$

Aksioma: $aS_1bcdeS_2fdS_2feacS_1$

Nosacījumi: $\{S_1 \to bcS_1,$

$S_2 \to feS_2\}$

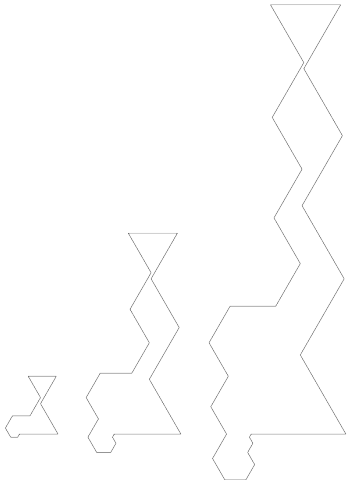


15. att. **($8k-5$)-stūru perfektie polimondi, kur** $k \geq 2$

Aksioma: $aS_1cbdfS_2edS_2efabS_1$

Nosacījumi: $\{S_1 \to cbS_1,$

$S_2 \to efS_2\}$

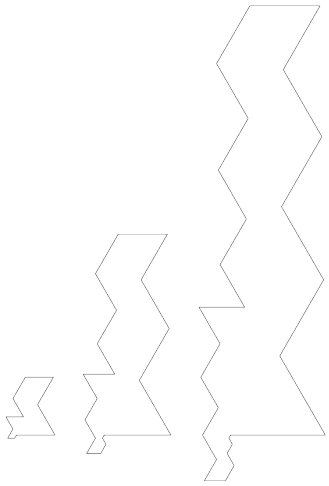


16. att. **($8k-5$)-stūru perfektie polimondi, kur** $k \geq 2$

Aksioma: $aS_1cbdeS_2fdS_2feabS_1$

Nosacījumi: $\{S_1 \to cbS_1,$

$S_2 \to feS_2\}$

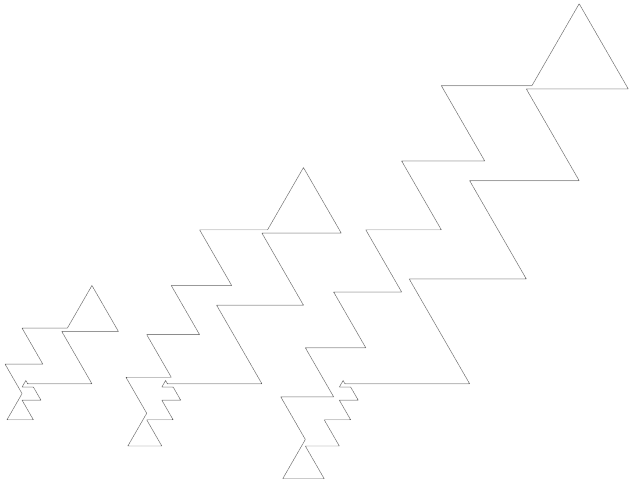


17. att. **($8k-5$)-stūru perfektie polimondi, kur** $k \geq 2$

Aksioma: $aS_1caceS_2dfdfeaS_1cacdbf$

Nosacījumi: $\{S_1 \to caS_1,$

$S_2 \to dfdfS_2\}$

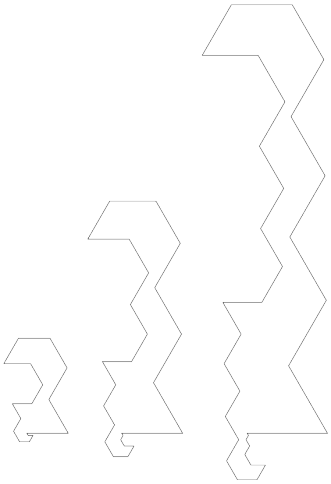


18. att. **($8k-7$)-stūru perfektie polimondi, kur** $k \geq 3$

Aksioma: $aS_1cbcdeaS_2fedS_2fefabdS_1c$

Nosacījumi: $\{S_1 \to cbS_1,$

$S_2 \to feS_2\}$



19. att. **($8k-7$)-stūru perfektie polimondi, kur** $k \geq 3$

Šīs ir visas līdz šim atrastās konstrukcijas ar indukcijas soli 8. Tomēr pietiek arī ar tikai četrām konstrukcijām, pa vienai katrā kategorijā ($8k-1$, $8k-3$, $8k-5$ un $8k-7$), jo tās pārklātu visus nepāra skaitļus. Jebkuram nepāra malu skaitam (sākot no 11), būtu shēma, kā izveidot perfektu polimondu.

Vēlāk tika atrastas arī konstrukcijas ar indukcijas soli 4, kas nozīmē, ka pietiek vien ar divām konstrukcijām, lai spētu uzkonstruēt polimondu katram nepāra malu skaitam (sākot no 17). Zemāk ir parādītas šo konstrukciju shēmas un šo konstrukciju pirmie trīs polimondi.

Aksioma: $acbS_1dfeS_2dfeS_2acS_3$

Nosacījumi: $\{ S_1 \to cT_1,$

$S_2 \to fT_2,$

$S_3 \to bT_3,$

$T_1 \to bS_1,$

$T_2 \to eS_2,$

$T_3 \to cS_3 \}$

20. att. **($4k-1$)-stūru perfektie polimondi, kur** $k \geq 3$

Aksioma: $abcS_1defS_2defS_2abS_3$

Nosacījumi: { $S_1 \to bT_1$,

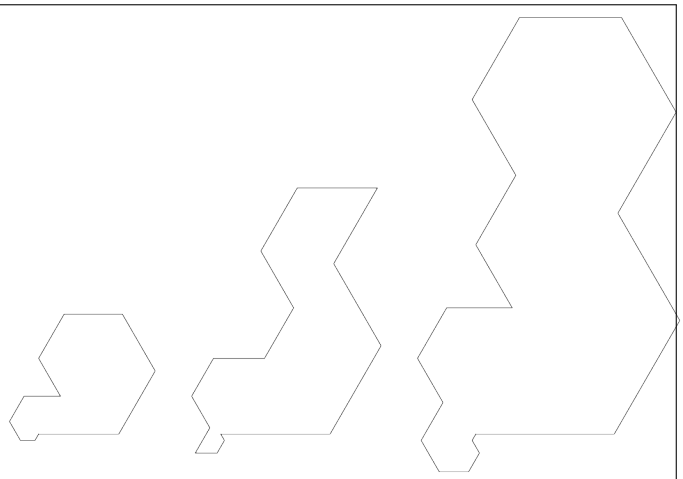
$S_2 \to eT_2$,

$S_3 \to cT_3$,

$T_1 \to cS_1$,

$T_2 \to fS_2$,

$T_3 \to bS_3$ }



21. att. **($4k-1$)-stūru perfektie polimondi, kur** $k \geq 3$

Aksioma: $acbcS_1deaefS_2dfefS_2abdbS_3$

Nosacījumi: { $S_1 \to bT_1$,

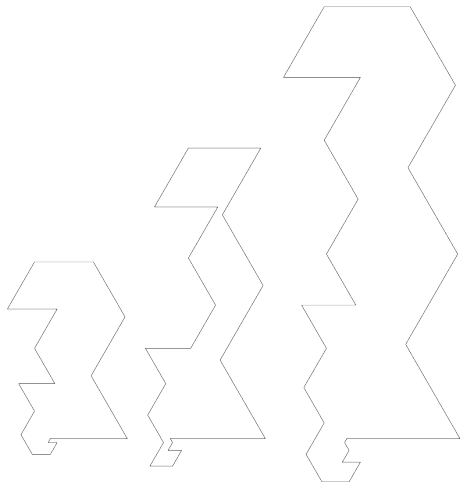
$S_2 \to eT_2$,

$S_3 \to cT_3$,

$T_1 \to cS_1$,

$T_2 \to fS_2$,

$T_3 \to bS_3$}



22. att. **($4k-3$)-stūru perfektie polimondi, kur** $k \geq 5$

SECINĀJUMI

Kursa darbā aplūkotie perfektie polimondi ir diezgan maz apskatīta tēma, tomēr perfektos polimondus ir vērts pētīt, jo ar viņiem var formulēt dažādus olimpiāžu, kā arī kombinatorikas un programmēšanas uzdevumus.

Iespējams, varētu arī atrast daudz dažādu sakarību, ja nedaudz paplašinātu perfekto polimondu klasi un apskatītu izogonus, tomēr tikai tos, kuri sevi nekrusto (drīkst pārklāties un pieskarties posmos). Šī izogonu apakšklase līdz šim vēl nav tikusi definēta un pētīta.

Ir atrastas vairākas dažādas metodes, kas veido perfektus polimondus jebkuram malu skaitam - gan pāra, gan nepāra. Lai gan visas šīs metodes, konstrukcijas līdz šim vienmēr ir devušas korektus polimondus, nav pilnībā pierādīts, ka tās tiešām ir pareizas, nav noliegts apgalvojums, ka kaut kad (varbūt bezgalīgi lielam malu skaitam) izveidosies polimonds, kas posmos sev pieskarsies vai sevi krustos, vai sevi pārklās.

IZMANTOTĀ LITERATŪRA UN AVOTI

[1] Elīna Buliņa. Maģiskie daudzstūri un to īpašības. *Bakalaura darbs*, 2016.

[2] Elīna Buliņa. Maģiskie polimino un polimondi. Vidzemes Augstskolas 11. studentu pētniecisko darbu konferences zinātnisko rakstu krājums:7., 8. lpp. 2017.

[3] Elīna Buliņa. Maģiskie polimondi un to īpašības. *Maģistra darbs*, 2018.

[4] Andrejs Cibulis un Elīna Buliņa. Fostering Teachers' Mathematical Competence in Problem Solving. *To be or not to be a great educator*:652—665, 2022.

[5] Alexander Keewatin Dewdney. An Odd Journey Along Even Roads Leads to Home in Golygon City. *Scientific American*, 263(1):118—121, 1990.

[6] Lee Sallows, Martin Gardner, Richard Kenneth Guy un Donald Knuth. Serial isogons of 90 degrees. *Mathematics Magazine*, 64(5):315—324, 1991.

PATEICĪBAS

Kursa darba autors izsaka lielu pateicību Latvijas Universitātes Eksakto zinātņu un tehnoloģiju fakultātes docentam Kalvim Apsītim par sniegto palīdzību darba izstrādē. Pielikumā pievienotie kodi tika veidoti kopīgi ar K. Apsīti, bet visi attēli, kuri parāda konstrukcijas pirmos trīs polimondus, tika veidoti ar K. Apsīša izveidoto datorprogrammu, kā arī konstrukciju aprakstīšana, izmantojot bezkonteksta gramatikas, ir K. Apsīša rezultāts.

PIELIKUMI

A. “BACKTRACKING” ALGORITMS

```

class Backtrack:

    # Backtracker objekts(nepabeigts šaha galdiņš, polimonds
    # vai kas cits).
    b = None

    # Konstruktors tikai iekopē padoto objektu
    def __init__(self, b):
        self.b = b

    # Mēģina risināt uzdevumu, atrodoties koka līmenī "level".
    # level=0 ir pašā augšā - pirms izdarīti "gājieni".
    def attempt(self, level):
        successful = False
        moveIterator = self.b.moves(level)

        for move in moveIterator:
            successful = False
            if self.b.valid(level, move):
                self.b.record(level, move)
                if self.b.done(level):
                    successful = True
                else:
                    successful = self.attempt(level+1)
                    if not successful:
                        self.b.undo(level, move)
            if successful:
                self.b.display()
                self.b.undo(level, move)
```

B. POLIMONDU PROBLĒMAS KODS

```
from backtrackk import *

class Polimondi:
    liss = []
    mysett = []

    def __init__(self, n):
        self.N = n
        self.liss.append([0, self.N])
        for i in range(1, self.N+1):
            self.mysett.append([i, 0, -i])

    # Funkcija 22.rindā pārbauda, vai nogrieznis ar garumu 1
    # nav novietots horizontāli.
    # Visa pārējā funkcija sataisa jaunu sarakstu (new_mysett_elem),
    # kur ir ieliktas iespējamās jauno malu koordinātas.
    # 47. rinda pārbauda, vai neviens no jaunajiem punktiem
    # jau neatrodas sarakstā mysett.
    def checks(self, move, level):
        self.level = level
        self.move = move
        if move[0] == 0 and (move[1] == 1 or move[1] == -1):
            return False
        new_mysett_elem = []
        ped = self.mysett[-1]
        ll0 = self.move[0]
        ll1 = self.move[1]
        for k in range(self.N-self.level):
            if ll0 == 0 and ll1 > 0:
```

```
                new_mysett_elem.append(
                    [(k+1+ped[0]), (ped[1]), (ped[2]-(k+1))])
            elif ll0 == 0 and ll1 < 0:
                new_mysett_elem.append(
                    [(ped[0]-(k+1)), (ped[1]), (k+1+ped[2])])
            elif ll0 > 0 and ll1 > 0:
                new_mysett_elem.append(
                    [(k+1+ped[0]), (ped[1])-(k+1), (ped[2])])
            elif ll0 < 0 and ll1 < 0:
                new_mysett_elem.append(
                    [(ped[0])-(k+1), (k+1+ped[1]), (ped[2])])
            elif ll0 < 0 and ll1 > 0:
                new_mysett_elem.append(
                    [(ped[0]), (k+1+ped[1]), (ped[2])-(k+1)])
            elif ll0 > 0 and ll1 < 0:
                new_mysett_elem.append(
                    [(ped[0]), (ped[1])-(k+1), (k+1+ped[2])])
        if all(a not in self.mysett for a in new_mysett_elem) == False:
            return False
        else:
            self.new_mysett_elem = new_mysett_elem
            # print(self.liss)
            # print(self.new_mysett_elem)
            # print(self.mysett)
            # print(" ")
            return True

    # Funkcija pārbauda, vai pievienojot jauno malu, ir iespējams
    # atgriezties atpakaļ sākumpunktā ar atlikušajām malām.
    # d-jau izveidotā polimonda augstums, v-platums.
    def checks2(self, move, level):
        self.level = level
        self.move = move
```

```
        self.list1 = self.liss.copy()
        self.list1.append(move)
        d = 0
        v = 0
        for k in self.list1:
            d += k[0]
            v += k[1]
        m = self.N - self.level - 1
        mm = (m*(m+1))/2
        # self.debugstate2(mm, v, d, m, self.level)
        if (mm < abs(d)) or (mm < abs(v)):
            return False
        return True

    def valid(self, level, move):
        self.p = move
        # self.debugstate(move, level)
        c1 = self.checks(move, level)
        c2 = self.checks2(move, level)
        # print("c1={}, c2={}".format(c1, c2))
        return c1 and c2

    def done(self, level):
        # if level == self.N - 1:
        #     return True
        # return False
        return (level == self.N - 1)

    # Pievienojam jauno malu sarakstā liss un šīs malas visus punktus
    # sarakstā mysett.
    def record(self, level, move):
        self.liss.append(move)
        for k in self.new_mysett_elem:
```

```
            self.mysett.append(k)

    # Parāpjamies atpakaļ, ja bija sasniegts strupceļš.
    def undo(self, level, move):
        self.liss = self.liss[:-1]
        self.mysett = self.mysett[:-(self.N-(level))]
        self.new_mysett_elem = []
        self.list1 = []

    # Izvada risinājumu.
    def display(self):
        print("{},".format(self.liss))

    # Atgriež četrus (pirmājā solī divus) iespējamos gājienus,
    # kā polimondu no dotā punkta var turpināt.
    def moves(self, level):
        self.level = level
        if self.level == 1:
            x111 = self.N - 1
            li1 = [[x111, -x111 / 2], [x111, x111 / 2]]
        else:
            x1 = self.N - self.level
            p = self.p
            if p[0] == 0 and p[1] > 0 or p[0] == 0 and p[1] < 0:
                li1 = [[x1, -x1 / 2], [x1, x1 / 2],
                       [-x1, -x1 / 2], [-x1, x1 / 2]]
            elif p[0] < 0 and p[1] > 0 or p[0] > 0 and p[1] < 0:
                li1 = [[0, -x1], [x1, x1 / 2], [0, x1], [-x1, -x1 / 2]]
            elif p[0] < 0 and p[1] < 0 or p[0] > 0 and p[1] > 0:
                li1 = [[0, -x1], [x1, -x1 / 2], [0, x1], [-x1, x1 / 2]]
        return li1

def main():
```

```
    q = Polimondi(N)
    b = Backtrack(q)
    if b.attempt(1):
        q.display()

if __name__ == '__main__':
    main()

```